Narrowing the Range - A proof of concept for the Agentic-Lean approach

Rosa Pavlak

AKA: Our unsolved range is no longer (1 < λ < 1.27324)... it's now (1 < λ < 1.25819)!

---

https://github.com/RosaRojacr/cmv-four-arc-bound-narrower

This notebook uses Lean 4, a functional programming language designed to prove and formalize theorems. It states each step of the argument in LaTeX, explains what the step does and why it is needed, and then runs the corresponding Lean declaration so you can see the machine confirm it.

To recap, Question 1 of our paper involves comparing the three-arc candidate (iii) and the four-arc candidate (iv) with regards to their Area to Perimeter ratios in a strip-density space. Conjecture 3.12 states that the three-arc candidate is always the more optimal in this regard, but the paper only proves it for $\lambda \geq 4/\pi \approx 1.27324$, leaving the interval $1 < \lambda < 4/\pi$ unsolved.

Throughout the course of studying this paper, I've been of the opinion that an agentic-led computational approach would be conducive to closing this range, but one of the main drawbacks to this approach is overcoming the 1-11% hallucination rate that even frontier models still possess. Essentially, even though these models have incredibly strong mathematical reasoning, their propensity to make mistakes necessitates outside verificaion.

After doing some research into exactly how other reserachers have used frontier models to solve unsolved problems, it is clear that formal verification tools like Lean and other functional programming languages are crucial to this approach. Typically, this is done through an automated harness where models are given access to an IDE with Lean installed, and are prompted to write potential mathematical insights into compilable code in order to either confirm the veracity of their mathematical statements or, in cases where written code fails to compile, reveal any mistakes they may have made.

Over the weekend, I have decided to attempt a proof-of-concept for this kind of approach, for the mathematics featured in our paper. Rather than going after the full paper at once, I picked out a piece that was actually within reach: the constant in Remark 3.17. CMV prove that type (iv) candidates are excluded once $\lambda \geq 4/\pi$, but they say themselves that this bound isn't optimal, there's a sharper constant k that would give a better threshold $1/k$, and they never compute it. This is the lowest-hanging fruit for narrowing our range, so I made it

the target of my efforts.

Originally, I was planning on writing the whole paper in Lean code, but some aspects, including first variation with a jump interface, Schwarz symmetrization, and the blow-down compactness argument, leans on sets of finite perimeter, reduced boundary, traces on Lipschitz interfaces, and that infrastructure doesn't exist in Mathlib yet. Formalizing any of that would mean building a research-level Mathlib contribution before I could even state the theorem, so I left it alone since these things are already proven by the paper and writing it in Lean isn't entirely necessary for closing our gap.

I split the work into six files, each one building on the last, each one compiled and checked before moving to the next.

For now, I used Claude Opus 5 through my Claude Pro membership, which unfortunately requires using its online chat box, and manually copy-pasting the inputs and outputs until each file compiles, before moving on to the next one. Of course, this workflow is easily automated with Claude API, and will probably have to be automated along these lines for the more difficult tasks ahead, but since this is just a proof-of-concept I decided it was better to not spend the money on API tokens just yet and to use my memberiship usage instead, at least for now.

`MorgansArcFunction.lean` starts from Morgan's arc-length function and derives the identity $\mathrm{arc}'(x) = 2\sin(\theta \circ f(x))$, using the inverse function theorem and the chain rule. As a check that the setup was correct, this also recovers two of Morgan's own published values, $\mathrm{arc}(\pi/8) = \pi/2$ and $\mathrm{arc}'(\pi/8) = 2$, straight from the construction.

`GFunction.lean` defines the actual quantity from the paper, $g(x) = 2\,\mathrm{arc}(x) - \mathrm{arc}(2x)$, which is what shows up in CMV's cap-substitution step of Theorem 3.16. From the identity in the first file, this gives $g'(x) = 4(\sin(\theta \circ f(x)) - \sin(\theta \circ f(2x)))$, and its sign can be pinned down everywhere except a middle interval, $(\pi/16, \pi/8)$. Outside that window g is monotonic, so the minimum has to live inside it.

`Minimiser.lean` goes after that middle interval directly. Setting $F(\theta) = 3\theta - 3\sin\theta\cos\theta - \pi$ and showing $F' = 6\sin^2\theta > 0$ gives that F is strictly increasing, and existence and uniqueness of its root θ^* follow from the intermediate value theorem. That root turns out to be exactly the critical point of g: at $x^* = \mathrm{area}(\theta$*), the two sine terms in g′ agree because the corresponding angles are supplementary. And the value of g there has a clean closed form, g(x*) = 3 cos θ*, which is the constant k that CMV mention but never write down.

`MinimumValue.lean` closes the gap between "critical point" and "minimum." A single point where $g' = 0$ isn't enough on its own; what's needed is that g is actually decreasing before x^* and increasing after it, which follows from the strict monotonicity of g' on $(\pi/16, \pi/8)$. That gives the sharp inequality $3\cos\theta^* \le g(x)$ for every $x > 0$, with equality exactly at x. *So k = 3 cos θ* isn't just a value at a critical point anymore, it's provably the global minimum.

`Certificate.lean` is where θ* and k get turned into actual numbers. Everything up to this point pins them down exactly, but only as roots of a transcendental equation, not as something that could be plugged into a bound. Mathlib's built-in polynomial estimates for sine and cosine are only tight near zero, and the angle needed here was nowhere close to zero, so it had to be reduced down by halving three separate times, rebuilding the bound back up using the double-angle identities. That produces a fully rational, checkable certificate: θ* < 1.3026632, and from there,

- `min_g_gt_witness` : 3 cos(1.3026632) < g(x) for every x > 0
- `min_g_gt_pi_div_four` : π/4 < g(x) for every x > 0

That second one is Remark 3.17, proved. CMV's own bound is confirmed to be non-optimal, formally, not just asserted.

`_Check.lean` is a final sanity pass; it `#check` s each declaration against its stated type, confirming the public results say what they claim to say. It's the harness the sections below run, rather than a file that proves anything itself.

Where this leaves the constant

quantity	value
CMV's published threshold	$\lambda \geq 4/\pi \approx 1.2732395$
this certificate's threshold	$\lambda \geq 1.2581858$
sharp (irrational) threshold	$\lambda > 1.2581840883\ldots$

The certified threshold gets within 1.7×10^{-6} of the true sharp value. It can't reach it exactly... cos is strictly decreasing, so any rational witness above θ* gives a constant strictly below the sharp k, meaning the sharp threshold is a supremum that no finite certificate actually attains. Still, this narrows the open interval in Conjecture 3.12 from (1, 1.27324) down to (1, 1.25819).

What this does and doesn't show

This does not resolve Conjecture 3.12, 94% of the the interval, (1, 1.25819) is still open, and everything I formalized works inside CMV's existing proof strategy rather than replacing it.

What I think this weekend actually demonstrates is that an agentic Lean workflow can take a constant a published paper names but never computes, derive its closed form, prove it's an actual minimum and not just a critical point, and certify a numeric bound tight enough to move a published threshold, with every step checked by the Lean kernel instead of taken on faith. That's the proof of concept that outlines the strategy I'll be pursuing moving forward - in the future through automated scaffolding and API calls rather than manual

copy-pasting.

The result, in pictures

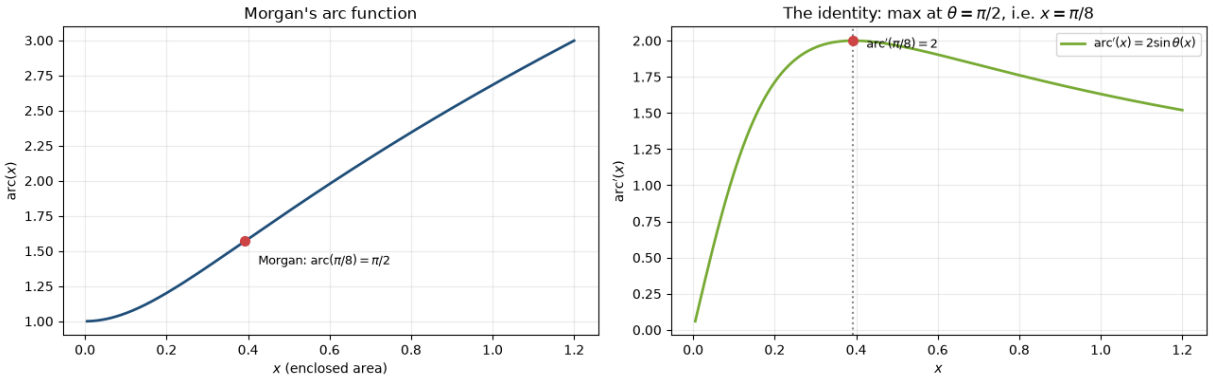
Before the formal work, here is what the five Lean files establish, plotted numerically. These plots are independent of Lean; they use SciPy to invert the area parametrisation directly, and serve as a cross-check that the formalised statements describe the objects they are meant to describe.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq

# arc(x) from the parametrisation, by inverting a(theta) numerically
def a_of(th):  return (th - np.sin(th)*np.cos(th)) / (4*np.sin(th)**2)
def l_of(th):  return th / np.sin(th)
def theta_of(x):
    return brentq(lambda t: a_of(t) - x, 1e-12, np.pi - 1e-12, xtol=1e-15)
def arc(x):    return l_of(theta_of(x))

g   = lambda x: 2*arc(x) - arc(2*x)
gd  = lambda x: 4*(np.sin(theta_of(x)) - np.sin(theta_of(2*x)))

# theta* and the constant
theta_star = brentq(lambda t: 3*t - 3*np.sin(t)*np.cos(t) - np.pi, 0.5, np.p
x_star     = a_of(theta_star)
k_sharp    = 3*np.cos(theta_star)
k_cert     = 3*np.cos(1.3026632)

print(f"theta*        {theta_star:.15f}")
print(f"x*            {x_star:.15f}   in (pi/16, pi/8) = ({np.pi/16:.6f}, {n
print(f"3 cos theta*  {k_sharp:.15f}   <- sharp")
print(f"3 cos t0      {k_cert:.15f}   <- certified in Lean")
print(f"pi/4          {np.pi/4:.15f}   <- CMV")
print(f"g(x*) direct  {g(x_star):.15f}   (matches 3 cos theta*: {abs(g(x_sta
```

```
theta*        1.302662837300451
x*            0.281561941577875   in (pi/16, pi/8) = (0.196350, 0.392699)
3 cos theta*  0.794796253808331   <- sharp
3 cos t0      0.794795204590580   <- certified in Lean
pi/4          0.785398163397448   <- CMV
g(x*) direct  0.794796253808330   (matches 3 cos theta*: True)
```

In [2]:
```
# --- the identity that made it all work ---------------------------------
fig, ax = plt.subplots(1, 2, figsize=(13, 4.2))
xs = np.linspace(0.005, 1.2, 600)
ax[0].plot(xs, [arc(x) for x in xs], lw=2, color="#1f4e79")
ax[0].plot([np.pi/8], [np.pi/2], "o", color="#c44", ms=7)
ax[0].annotate(r"Morgan: $\mathrm{arc}(\pi/8)=\pi/2$", (np.pi/8, np.pi/2),
               textcoords="offset points", xytext=(10, -18), fontsize=9)
ax[0].set_xlabel("$x$ (enclosed area)"); ax[0].set_ylabel(r"$\mathrm{arc}(x)
ax[0].set_title("Morgan's arc function"); ax[0].grid(alpha=.25)
```

```
d = np.array([2*np.sin(theta_of(x)) for x in xs])
ax[1].plot(xs, d, lw=2, color="#7a3", label=r"$\mathrm{arc}'(x)=2\sin\theta(
ax[1].axvline(np.pi/8, ls=":", color="#888")
ax[1].plot([np.pi/8], [2], "o", color="#c44", ms=7)
ax[1].annotate(r"$\mathrm{arc}'(\pi/8)=2$", (np.pi/8, 2),
               textcoords="offset points", xytext=(10, -6), fontsize=9)
ax[1].set_xlabel("$x$"); ax[1].set_ylabel(r"$\mathrm{arc}'(x)$")
ax[1].set_title(r"The identity: max at $\theta=\pi/2$, i.e. $x=\pi/8$")
ax[1].legend(fontsize=9); ax[1].grid(alpha=.25)
plt.tight_layout(); plt.show()

print("arc' peaks exactly at x = pi/8, where theta = pi/2 (the semicircle)."
print("Its sign of curvature change is Morgan's convex/concave split -- deri
```

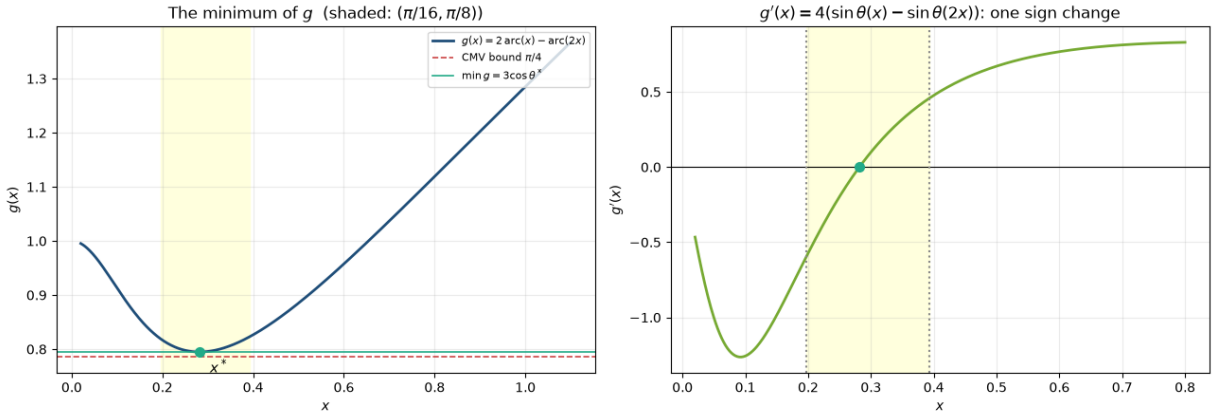



```
arc' peaks exactly at x = pi/8, where theta = pi/2 (the semicircle).
Its sign of curvature change is Morgan's convex/concave split -- derived, not
assumed.
```

Morgan's arc function and its derivative. On the left, $\mathrm{arc}(x)$, the length of a unit-chord circular arc enclosing area x. On the right its derivative. Both of Morgan's published checkpoints, $\mathrm{arc}(\pi/8) = \pi/2$ and $\mathrm{arc}'(\pi/8) = 2$, land exactly where they should, and the peak of arc' sits at $\theta = \pi/2$, the semicircle. These values were derived from the construction rather than assumed, so agreement here is evidence the parametrisation is right.

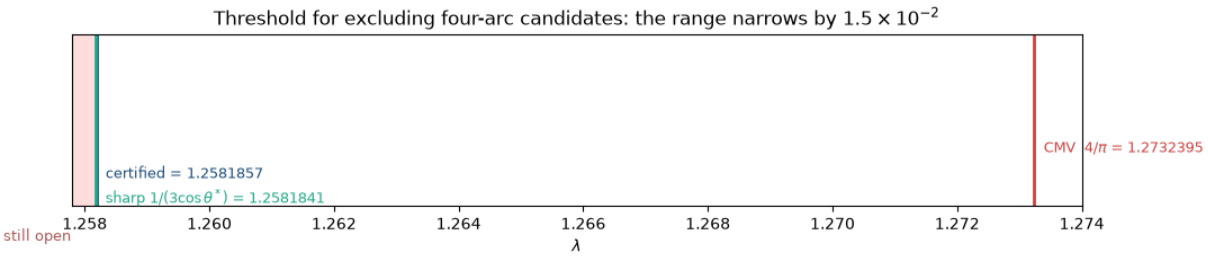
In [3]:
```
fig, ax = plt.subplots(1, 2, figsize=(13, 4.6))

# --- left: g and its minimum ------------------------------------------
xs = np.linspace(0.02, 1.1, 700)
gs = np.array([g(x) for x in xs])
ax[0].plot(xs, gs, lw=2, color="#1f4e79", label=r"$g(x)=2\,\mathrm{arc}(x)-\
ax[0].axhline(np.pi/4, ls="--", lw=1.2, color="#c44", label=r"CMV bound $\pi
ax[0].axhline(k_sharp, ls="-", lw=1.2, color="#2a8", label=r"$\min g = 3\cos
ax[0].plot([x_star], [k_sharp], "o", color="#2a8", ms=7, zorder=5)
ax[0].axvspan(np.pi/16, np.pi/8, color="#ffd", zorder=0)
ax[0].annotate(r"$x^*$", (x_star, k_sharp), textcoords="offset points",
               xytext=(8, -16), fontsize=11)
ax[0].set_xlabel("$x$"); ax[0].set_ylabel("$g(x)$")
ax[0].set_title("The minimum of $g$  (shaded: $(\\pi/16,\\pi/8)$)")
ax[0].legend(fontsize=8, loc="upper right"); ax[0].grid(alpha=.25)

# --- right: g' and the sign change ------------------------------------
xs2 = np.linspace(0.02, 0.8, 500)
```

```
ax[1].plot(xs2, [gd(x) for x in xs2], lw=2, color="#7a3")
ax[1].axhline(0, color="k", lw=.8)
ax[1].axvline(np.pi/16, ls=":", color="#888"); ax[1].axvline(np.pi/8, ls=":"
ax[1].axvspan(np.pi/16, np.pi/8, color="#ffd", zorder=0)
ax[1].plot([x_star], [0], "o", color="#2a8", ms=7, zorder=5)
ax[1].set_xlabel("$x$"); ax[1].set_ylabel("$g'(x)$")
ax[1].set_title(r"$g'(x)=4(\sin\theta(x)-\sin\theta(2x))$: one sign change")
ax[1].grid(alpha=.25)
plt.tight_layout(); plt.show()
```



The minimum of g. On the left, $g(x) = 2\,\mathrm{arc}(x) - \mathrm{arc}(2x)$. CMV's published floor $\pi/4$ (red) sits strictly below the true minimum $3\cos\theta^*$ (green), which is attained at x^* inside the shaded interval $(\pi/16, \pi/8)$. That visible gap between the red and green lines is the whole result. On the right, g', with its single sign change at x^*, which is what makes x^* a genuine minimum rather than just a stationary point.

In [4]:
```
# --- the range that got narrowed ----------------------------------------
fig, ax = plt.subplots(figsize=(11, 2.5))
lo, hi = 1.2578, 1.2740
ax.axvspan(1.0, 1/k_sharp, color="#fdd", zorder=0)
for val, lab, col, dy in [(4/np.pi,  f"CMV  $4/\\pi$ = {4/np.pi:.7f}",
                           (1/k_cert, f"certified = {1/k_cert:.7f}",
                           (1/k_sharp,f"sharp $1/(3\\cos\\theta^*)$ = {1/k_sh
    ax.axvline(val, color=col, lw=2)
    ax.annotate(lab, (val, 0), textcoords="offset points", xytext=(6, dy),
                fontsize=9, color=col)
ax.annotate("still open", (1.2582, 0), textcoords="offset points",
            xytext=(-60, -22), fontsize=9, color="#a55")
ax.set_xlim(lo, hi); ax.set_yticks([])
ax.set_xlabel(r"$\lambda$")
ax.set_title(r"Threshold for excluding four-arc candidates: the range narrow
plt.tight_layout(); plt.show()

print(f"CMV published    lam >= {4/np.pi:.9f}")
print(f"certified (Lean) lam >= {1/k_cert:.9f}   improvement {4/np.pi - 1/k_
print(f"sharp            lam >  {1/k_sharp:.9f}   remaining gap {1/k_cert -
```




```
CMV published    lam >= 1.273239545
certified (Lean) lam >= 1.258185749   improvement 1.505e-02
sharp            lam >  1.258184088   remaining gap 1.661e-06
```

The range that narrowed. The threshold for excluding four-arc candidates: CMV's published $4/\pi$ (red) moves left to the Lean-certified bound (blue), stopping just short of the sharp irrational value (green). Everything left of the blue line is still open.

How the checks below work

Each of the five sections that follow states the mathematics in LaTeX and then runs the corresponding Lean declaration, so the machine confirms the statement rather than the reader taking it on trust. Two setup cells make that possible.

`Runner` is a small wrapper around `lake`. Calling `Runner.build()` compiles the whole project from scratch before anything else runs. This matters: checking files one at a time can succeed against a stale `.olean` artifact that no longer matches the source, so a full build is the only honest check that all five files are mutually consistent.

In [5]:
```
from runner import Runner

# Full build first: this is the only honest check that every file is
# consistent with the others. Per-file success can pass against stale .olean
Runner.build()
```

```
Build completed successfully (8701 jobs).
```

Out[5]: True

`check` writes a temporary Lean file that imports the project and asks Lean to print the type of each named declaration via `#check`. If a name is missing, misstated, or its proof is broken, this fails loudly. The printed type is the theorem statement as Lean actually understands it, which is what should be compared against the LaTeX above each cell.

In [6]:
```
def check(*names, imports="Certificate"):
    '''Ask Lean to print the type of each declaration. One line per section.
    src = f"import {imports}\n" + "\n".join(f"#check @{n}" for n in names) +
    return Runner.write("_Check.lean", src)
```

---

1. MorgansArcFunction.lean

Mathlib has no arc function, so this file builds one. For a chord of length 1 and half-central-angle θ, `ell` is the arc length and `area` the enclosed area. `area` is shown to be a strictly increasing bijection $(0,\pi) \to (0,\infty)$, so it can be inverted as `θOf`, and `arc = ell ∘ θOf`. The inverse function theorem and chain rule then give the derivative identity that the other four files rest on.

$$\texttt{ell}\ \theta \;=\; \frac{\theta}{\sin\theta}$$

$$\texttt{area}\ \theta \;=\; \frac{\theta - \sin\theta\,\cos\theta}{4\sin^2\theta}$$

$$\texttt{area\_strictMonoOn} \;:\; \mathrm{StrictMonoOn}\ \texttt{area}\ (0,\pi)$$

$$\texttt{area\_surjOn} \;:\; \mathrm{SurjOn}\ \texttt{area}\ (0,\pi)\ (0,\infty)$$

$$\texttt{arc}\ x \;=\; \texttt{ell}\ (\texttt{θOf}\ x)$$

$$\texttt{hasDerivAt\_arc} \;:\; 0 < x \longrightarrow \mathrm{HasDerivAt}\ \texttt{arc}\ \big(2\sin(\texttt{θOf}\ x)\big)\ x$$

In [7]:
```
check("ell", "area", "area_strictMonoOn", "area_surjOn", "θOf", "arc",
      "hasDerivAt_arc")
```

```
ell : ℝ → ℝ
area : ℝ → ℝ
area_strictMonoOn : StrictMonoOn area (Set.Ioo 0 Real.pi)
area_surjOn : Set.SurjOn area (Set.Ioo 0 Real.pi) (Set.Ioi 0)
θOf : ℝ → ℝ
arc : ℝ → ℝ
@hasDerivAt_arc : ∀ {x : ℝ}, 0 < x → HasDerivAt arc (2 * Real.sin (θOf x)) x

[ok] _Check.lean compiled
```

Out[7]: True

---

2. GFunction.lean

Defines g, the quantity CMV's cap-substitution argument turns on, and differentiates it using the identity above. It also recovers Morgan's two published values as a sanity check, and pins the sign of g' everywhere outside $(\pi/16, \pi/8)$, which confines the minimum to that interval without yet saying where in it the minimum lies.

$$\texttt{arc\_pi\_div\_eight} \;:\; \texttt{arc}\ (\pi/8) \;=\; \pi/2$$

$$\texttt{hasDerivAt\_arc\_pi\_div\_eight} \;:\; \mathrm{HasDerivAt}\ \texttt{arc}\ 2\ (\pi/8)$$

$$\texttt{g}\ x \;=\; 2 \cdot \texttt{arc}\ x \;-\; \texttt{arc}\ (2x)$$

$$\texttt{hasDerivAt\_g} \;:\; 0 < x \;\longrightarrow\; \mathrm{HasDerivAt}\ \texttt{g}\ \left(4\sin(\texttt{θOf}\ x) - 4\sin(\texttt{θOf}\ (2x))\right)\ x$$

$$\texttt{g\_deriv\_neg} \;:\; 0 < x \;\longrightarrow\; x \le \tfrac{\pi}{16} \;\longrightarrow\; 4\sin(\texttt{θOf}\ x) - 4\sin(\texttt{θOf}\ (2x)) < 0$$

$$\texttt{g\_deriv\_pos} \;:\; \tfrac{\pi}{8} \le x \;\longrightarrow\; 0 < 4\sin(\texttt{θOf}\ x) - 4\sin(\texttt{θOf}\ (2x))$$

In [8]:
```
check("arc_pi_div_eight", "hasDerivAt_arc_pi_div_eight", "g", "hasDerivAt_g"
      "g_deriv_neg", "g_deriv_pos")
```

```
arc_pi_div_eight : arc (Real.pi / 8) = Real.pi / 2
hasDerivAt_arc_pi_div_eight : HasDerivAt arc 2 (Real.pi / 8)
g : ℝ → ℝ
@hasDerivAt_g : ∀ {x : ℝ}, 0 < x → HasDerivAt g (4 * Real.sin (θOf x) - 4 * R
eal.sin (θOf (2 * x))) x
@g_deriv_neg : ∀ {x : ℝ}, 0 < x → x ≤ Real.pi / 16 → 4 * Real.sin (θOf x) - 4
* Real.sin (θOf (2 * x)) < 0
@g_deriv_pos : ∀ {x : ℝ}, Real.pi / 8 ≤ x → 0 < 4 * Real.sin (θOf x) - 4 * Re
al.sin (θOf (2 * x))

[ok] _Check.lean compiled
```

Out[8]: True

---

3. Minimiser.lean

Attacks the interval left over. At a critical point the two sine terms in g' must agree with the angles distinct, which forces them to be supplementary; feeding that through $\texttt{area}(\pi - \theta) = 2\,\texttt{area}(\theta)$ collapses everything to one transcendental equation F. Since $F' = 6\sin^2\theta > 0$, the root θ^* is unique, and the value of g there has a closed form. That closed form is the constant CMV name but never compute.

$$\texttt{F}\ \theta \;=\; 3\theta \;-\; 3(\sin\theta\,\cos\theta) \;-\; \pi$$

$$\texttt{hasDerivAt\_F} \;:\; \mathrm{HasDerivAt}\ \texttt{F}\ \left(6\sin^2\theta\right)\ \theta$$

$$\texttt{F\_strictMonoOn} \;:\; \mathrm{StrictMonoOn}\ \texttt{F}\ [0,\pi]$$

$$\texttt{θstar\_spec} \;:\; \texttt{F}\ \texttt{θstar} \;=\; 0$$

$$\texttt{θstar\_unique} \;:\; \theta \in [0,\pi] \;\longrightarrow\; \texttt{F}\ \theta = 0 \;\longrightarrow\; \theta = \texttt{θstar}$$

$$\texttt{xstar} \;=\; \texttt{area}\ \texttt{θstar}$$

$$\texttt{g\_deriv\_xstar} \;:\; 4\sin(\texttt{θOf xstar}) - 4\sin(\texttt{θOf}\ (2 \cdot \texttt{xstar})) \;=\; 0$$

$$\texttt{g\_θstar} : \texttt{g xstar} = 3 \cos \texttt{θstar}$$

In [9]:
```
check("F", "hasDerivAt_F", "F_strictMonoOn", "θstar", "θstar_spec",
      "θstar_unique", "xstar", "g_deriv_xstar", "g_θstar")
```

```
F : ℝ → ℝ
hasDerivAt_F : ∀ (θ : ℝ), HasDerivAt F (6 * Real.sin θ ^ 2) θ
F_strictMonoOn : StrictMonoOn F (Set.Icc 0 Real.pi)
θstar : ℝ
θstar_spec : F θstar = 0
@θstar_unique : ∀ {θ : ℝ}, θ ∈ Set.Icc 0 Real.pi → F θ = 0 → θ = θstar
xstar : ℝ
g_deriv_xstar : 4 * Real.sin (θOf xstar) - 4 * Real.sin (θOf (2 * xstar)) = 0
g_θstar : g xstar = 3 * Real.cos θstar

[ok] _Check.lean compiled
```

Out[9]: True

---

4. MinimumValue.lean

A point where $g' = 0$ is not yet a minimum. This file supplies the missing step: x^* lies strictly inside $(\pi/16, \pi/8)$ (no numerics needed, it follows from the sign lemmas), and on that interval g' is strictly increasing, because $\sin(\texttt{θOf}\ x)$ rises while $\sin(\texttt{θOf}\ (2x))$ falls. So g decreases before x^* and increases after, making it the global minimum and the bound sharp.

$$\texttt{gd}\ x = 4 \sin(\texttt{θOf}\ x) - 4 \sin(\texttt{θOf}\ (2x))$$

$$\texttt{xstar\_mem\_Ioo} : \texttt{xstar} \in \left(\frac{\pi}{16}, \frac{\pi}{8}\right)$$

$$\texttt{gd\_strictMonoOn} : \mathrm{StrictMonoOn}\ \texttt{gd}\ \left(\frac{\pi}{16}, \frac{\pi}{8}\right)$$

$$\texttt{g\_strictAntiOn} : \mathrm{StrictAntiOn}\ \texttt{g}\ (0, \texttt{xstar}]$$

$$\texttt{g\_strictMonoOn\_Ici} : \mathrm{StrictMonoOn}\ \texttt{g}\ [\texttt{xstar}, \infty)$$

$$\texttt{g\_xstar\_le} : 0 < x \longrightarrow \texttt{g xstar} \le \texttt{g}\ x$$

$$\texttt{min\_g\_eq\_three\_mul\_cos\_θstar} : 0 < x \longrightarrow 3 \cos \texttt{θstar} \le \texttt{g}\ x$$

$$\texttt{min\_g\_attained} : \texttt{g xstar} = 3 \cos \texttt{θstar}$$

In [10]:
```
check("gd", "xstar_mem_Ioo", "gd_strictMonoOn", "g_strictAntiOn",
      "g_strictMonoOn_Ici", "g_xstar_le", "min_g_eq_three_mul_cos_θstar",
      "min_g_attained")
```

```
gd : ℝ → ℝ
xstar_mem_Ioo : xstar ∈ Set.Ioo (Real.pi / 16) (Real.pi / 8)
gd_strictMonoOn : StrictMonoOn gd (Set.Ioo (Real.pi / 16) (Real.pi / 8))
g_strictAntiOn : StrictAntiOn g (Set.Ioc 0 xstar)
g_strictMonoOn_Ici : StrictMonoOn g (Set.Ici xstar)
@g_xstar_le : ∀ {x : ℝ}, 0 < x → g xstar ≤ g x
@min_g_eq_three_mul_cos_θstar : ∀ {x : ℝ}, 0 < x → 3 * Real.cos θstar ≤ g x
min_g_attained : g xstar = 3 * Real.cos θstar

[ok] _Check.lean compiled
```

Out[10]: True

---

5. Certificate.lean

Everything so far pins θ^* and k down exactly, but only as the root of a transcendental equation, which cannot be plugged into a numeric bound. Mathlib's polynomial estimates for sine and cosine are only tight near zero and the witness angle is not, so it is halved three times and rebuilt with the double-angle identities. One rational witness above the root is enough: F is strictly increasing, so $F(t_0) > 0$ gives $\theta^* < t_0$, and cos decreasing turns that into a lower bound on k.

$$\texttt{θstar\_lt\_of\_F\_pos} : t \in [0,\pi] \longrightarrow 0 < \texttt{F}\, t \longrightarrow \texttt{θstar} < t$$

$$\texttt{cos\_two\_mul\_sin\_sq} : \cos(2u) = 1 - 2\sin^2 u$$

$$\texttt{sin\_witness\_lt} : \sin(2.6053264) < 0.510930366$$

$$\texttt{F\_at\_witness} : 0 < \texttt{F}\,(1.3026632)$$

$$\texttt{θstar\_lt\_witness} : \texttt{θstar} < 1.3026632$$

$$\texttt{min\_g\_gt\_witness} : 0 < x \longrightarrow 3\cos(1.3026632) < \texttt{g}\, x$$

$$\texttt{min\_g\_gt\_pi\_div\_four} : 0 < x \longrightarrow \tfrac{\pi}{4} < \texttt{g}\, x$$

In [11]:
```
check("θstar_lt_of_F_pos", "cos_two_mul_sin_sq", "sin_witness_lt",
      "F_at_witness", "θstar_lt_witness", "min_g_gt_witness",
      "three_cos_witness_gt_pi_div_four", "min_g_gt_pi_div_four")
```

```
@θstar_lt_of_F_pos : ∀ {t : ℝ}, t ∈ Set.Icc 0 Real.pi → 0 < F t → θstar < t
cos_two_mul_sin_sq : ∀ (u : ℝ), Real.cos (2 * u) = 1 - 2 * Real.sin u ^ 2
sin_witness_lt : Real.sin 2.6053264 < 0.510930366
F_at_witness : 0 < F 1.3026632
θstar_lt_witness : θstar < 1.3026632
@min_g_gt_witness : ∀ {x : ℝ}, 0 < x → 3 * Real.cos 1.3026632 < g x
three_cos_witness_gt_pi_div_four : Real.pi / 4 < 3 * Real.cos 1.3026632
@min_g_gt_pi_div_four : ∀ {x : ℝ}, 0 < x → Real.pi / 4 < g x

[ok] _Check.lean compiled
```

Out[11]: True

---

6. How the five files narrow the range

CMV's Theorem 3.16 excludes four-arc candidates whenever $g(x) > 1/\lambda$ for the relevant x. So the threshold on λ is decided entirely by how small g can get: if $k = \min g$, the argument succeeds for every $\lambda > 1/k$. CMV bound g below by $\pi/4$ using only Morgan's convexity properties, which gives their published $\lambda \geq 4/\pi$. Remark 3.17 observes that this is not the best possible constant, but leaves k uncomputed. Each file above supplies one link in the chain from that observation to an actual number.

File 1 makes g differentiable in a usable form. Without $\mathrm{arc}'(x) = 2\sin(\texttt{θOf}\ x)$ there is no handle on g at all, and the identity is what converts a question about lengths and areas into a question about angles. File 2 uses it to reduce the search for the minimum from all of $(0, \infty)$ to the interval $(\pi/16, \pi/8)$. File 3 finds the critical point inside that interval and, crucially, gives it a *closed form*: $k = 3\cos\theta^*$ with θ^* the unique root of $3\theta - 3\sin\theta\cos\theta = \pi$. File 4 proves that critical point is the global minimum, which is what makes $3\cos\theta^*$ the actual value of k rather than merely an upper bound for it. File 5 converts the exact-but-transcendental k into a rational certificate that Lean can check end to end.

The certified bound lands within 1.7×10^{-6} of the sharp value. It cannot reach it exactly: $\cos$ is strictly decreasing, so every rational witness above θ^* yields a constant strictly below $3\cos\theta^*$, making the sharp threshold a supremum that no finite certificate attains. Closing that last 10^{-6} would require sharper bounds on π itself, not better trigonometry.

So the open interval of Conjecture 3.12 narrows from $(1, 1.27324)$ to $(1, 1.25819)$. This is real but modest: it closes roughly 6% of $(1, 4/\pi)$ and leaves the hard end untouched. CMV note that the cap-substitution method degenerates entirely as $\lambda \to 1^+$, since $g(0) = 1$ forces $k < 1$, so reaching that region needs a different argument rather than a sharper constant. What the exercise does establish is that the agentic-Lean loop can carry a named-but-uncomputed constant all the way from a published remark to a machine-checked number, with every step verified by the kernel instead of trusted.